Docker:

In this project we containerize a static web front-end application using the Docker platform. The application is served by NGINX, and the Dockerfile starts with the FROM nginx:stable-alpine base image, which offers a lightweight and efficient runtime environment ideal for static content. The COPY . /usr/share/nginx/html/ command transfers the compiled static assets into the default NGINX document root, and the RUN chown … && find … chmod instructions update permissions so that the nginx user can safely serve the files. Exposing port 80 allows the container to accept HTTP traffic with minimal setup. By using this container approach we ensure that the application behaves consistently across development, testing and production systems.

This containerised model simplifies deployment, enhances portability, and supports rapid, repeatable releases. Instead of managing multiple servers or environments, we now build the application once and run it the same way everywhere. The choice of NGINX and a minimal Alpine-based image keeps the runtime footprint low, reduces startup time and aligns with best-practices for serving static content in containers. Following this approach integrates naturally with the DevOps pipeline and helps avoid “it works on my machine” issues.

Installation of Docker:

1. Check Prerequisites

Before installing Docker, ensure that:

- The system supports virtualization (enable it in BIOS or UEFI if required).
- You have administrative privileges to install software.
- The operating system is one of the supported platforms such as Windows, macOS, or Linux.

2. Download Docker

Go to the official Docker website:

https://www.docker.com/get-started

Choose the Docker Desktop installer suitable for your operating system:

- **Windows:** Docker Desktop for Windows (.exe)
- **macOS:** Docker Desktop for macOS (.dmg)

- **Linux:** Install Docker Engine using your package manager or follow the installation guide for your distribution.

3. Install Docker

Depending on your platform:

- **Windows:** Run the Docker Desktop installer (.exe) and follow the setup wizard.
- **macOS:** Open the Docker.dmg file and drag Docker to the Applications folder.
- **Linux:** Use the command line to install Docker Engine. Example:

  sudo apt install docker-ce docker-ce-cli containerd.io

4. Start Docker Service

After installation, ensure that Docker is running before executing any commands.

- **Windows/macOS:** Launch Docker Desktop. The Docker icon will appear in the system tray or menu bar.

- **Linux:** Start Docker using the command:
  sudo systemctl start docker
- To verify Docker is active, run:

  docker --version

5. Verify Installation

- To confirm that Docker is installed and functioning correctly, open a terminal or command prompt and run:

  docker run hello-world

- If successful, you'll see a message stating that the "Hello from Docker!" container has run correctly. This confirms that the Docker Engine and container runtime are working properly.

6. Configure Permissions (Linux Only)

On Linux systems, you may need to configure permissions to run Docker commands without using sudo each time:

sudo usermod -aG docker $USER

Then log out and log back in to apply the changes.

7. Verify Docker Hub Access

Check if you can access Docker Hub by running:

docker login

Enter your credentials.

Creating Dockerfile and Building the Docker Image

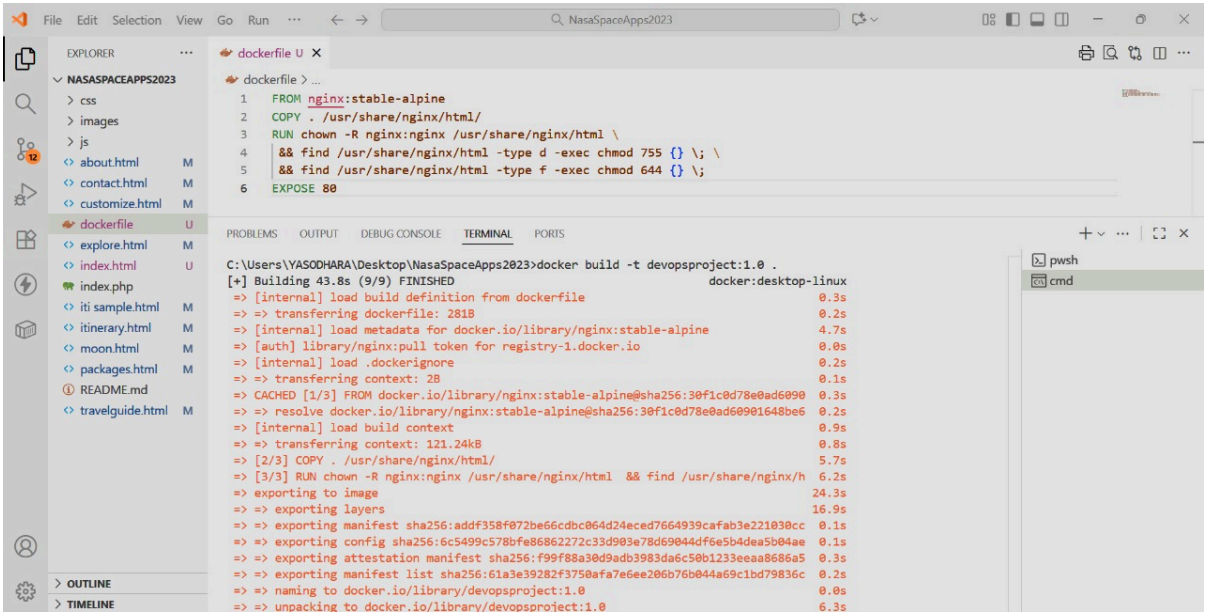
1. Create a Dockerfile

A Dockerfile is a text file that contains all the commands required to assemble a Docker image. It defines how the application will run inside a container and what dependencies it needs.

In this project, the Dockerfile is created in the root directory of the application with the following contents:

```
FROM nginx:stable-alpine

COPY . /usr/share/nginx/html/

RUN chown -R nginx:nginx /usr/share/nginx/html \

 && find /usr/share/nginx/html -type d -exec chmod 755 {} \; \

 && find /usr/share/nginx/html -type f -exec chmod 644 {} \;

EXPOSE 80
```

The above Dockerfile uses the official NGINX image as the base, copies the static web files into the NGINX HTML directory, and sets proper permissions to ensure that NGINX can serve them securely. The container will listen on port 80, which is the standard HTTP port.



2. Build the Docker Image

After creating the Dockerfile, open a terminal in the directory containing it and run the following command to build the image:

```
docker build -t my-docker-app .
```

Here,

- docker build initiates the image creation process.
- -t my-docker-app assigns a tag name to the image.
- . indicates the current directory as the build context.

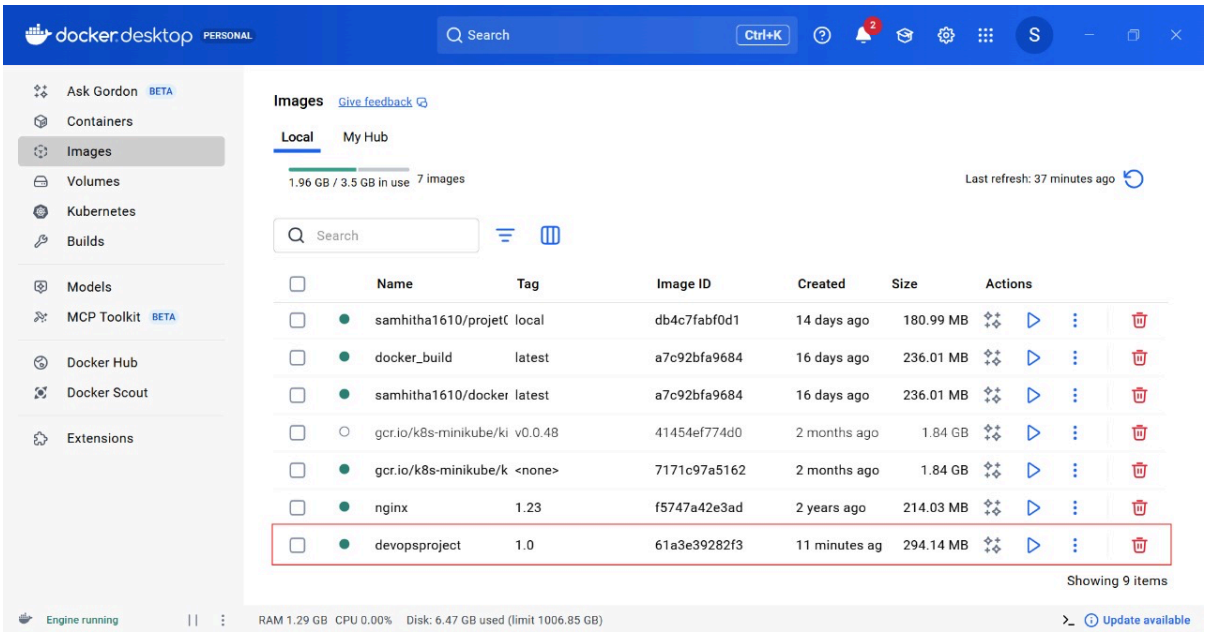
Once the build completes successfully, Docker packages all required files into an image that can be reused or shared via Docker Hub.

```
C:\Users\YASODHARA\Desktop\NasaSpaceApps2023>docker run -d --name devproject -p 8000:80 devopsproject:1.0
ca042da8d0f087ae73793b1113ba83e2d46954e63257c4a30272d2beab7c4f42
```

3. Verify the Image

To confirm that the image has been built successfully, run:

```
docker images
```



Running the Docker Container

1. Run the Docker Container

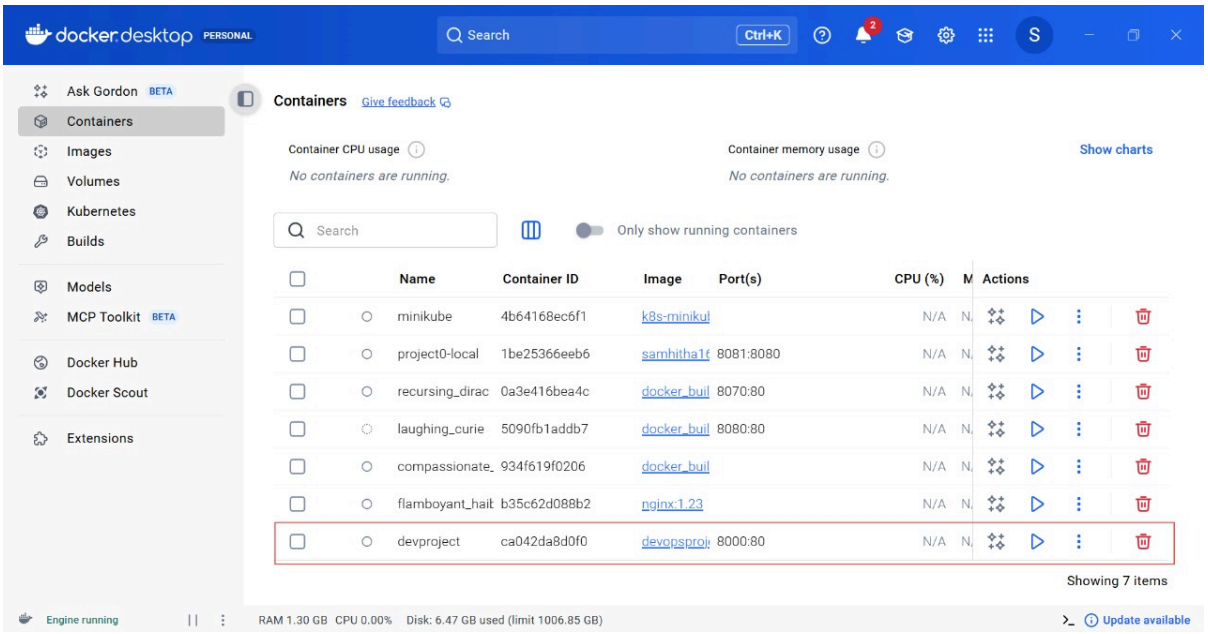
Once the Docker image is successfully built, the next step is to run it inside a container. Use the following command to start a new container based on the image you created:

```
docker run -d -p 8080:80 --name my-docker-container my-docker-app
```

Explanation:

- -d runs the container in detached mode (in the background).
- -p 8080:80 maps port 8080 on the host machine to port 80 inside the container.
- --name my-docker-container assigns a readable name to the container.
- my-docker-app is the image name built in the previous step.

Once executed, Docker starts the NGINX server within the container, hosting your static website.



2. Verify the Running Container

Check whether the container is running by using the following command:

```
docker ps
```

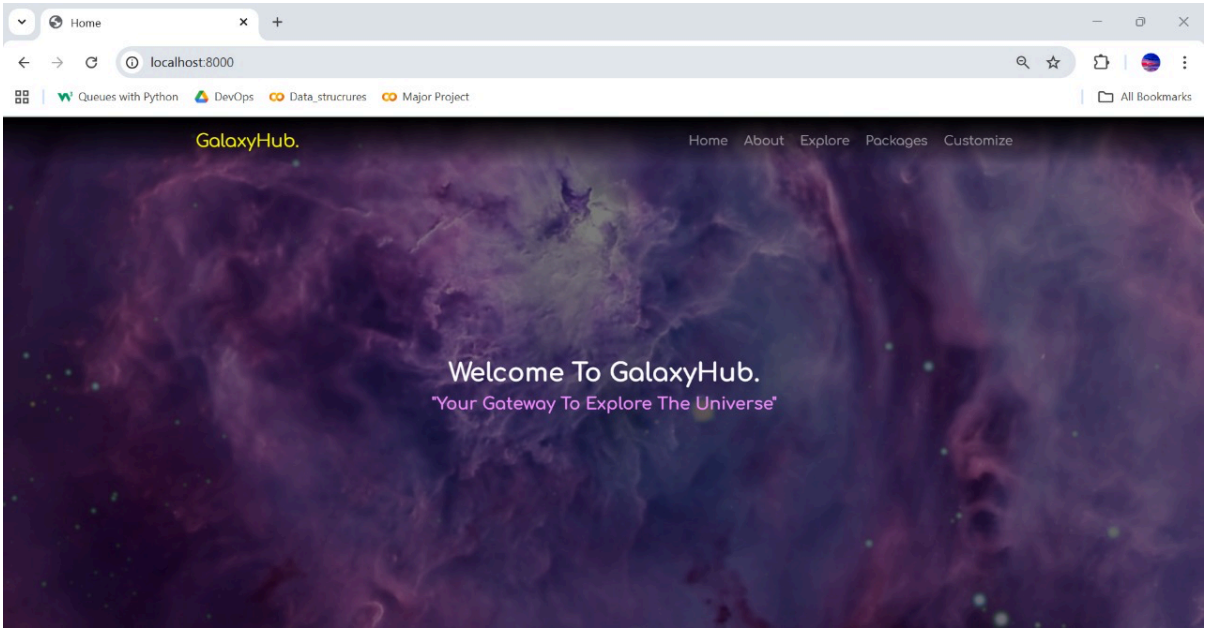
This command lists all active containers. You should see your container listed with the image name, container ID, and port mapping (for example, 0.0.0.0:8080->80/tcp).

3. Access the Application

Once the container is running, open any web browser and enter the following URL:

http://localhost:8080

This should load the application hosted by the NGINX server inside the Docker container. If configured correctly, the webpage will display the static content that was copied into the /usr/share/nginx/html directory during the build.



Conclusion

In this part of the project, Docker was implemented to containerize the application and ensure consistent performance across different environments. By using a Dockerfile based on the lightweight NGINX image, the application was packaged together with all its required files and dependencies, eliminating manual configuration and deployment issues.

Through this implementation, we achieved several key benefits:

- **Portability:** The Docker image can run on any system that has Docker installed, regardless of the underlying operating system.

- **Scalability:** Multiple containers can be deployed easily, allowing horizontal scaling of the application.

- **Efficiency:** The use of an Alpine-based NGINX image ensures minimal resource consumption and faster container startup times.

- **Reproducibility:** Every environment, from development to production, runs the same container image, preventing configuration drift.

With Docker successfully integrated, the deployment process has become faster, more reliable, and easier to manage. This containerized setup also provides a strong foundation for further automation and orchestration, which will be handled in the next phase of the project using Kubernetes.